



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Michael Roman Alavez Colli

Fecha:
20 de Julio 2026

Tarea 3: Segundo Parcial

1. Introducción

El presente documento tiene como objetivo analizar el caso de estudio del sistema **GymManager**, una plataforma web desarrollada para administrar un gimnasio mediante módulos de clientes, membresías, pagos, entrenadores, reservación de clases, reportes administrativos y control de asistencia.

Durante el desarrollo del sistema se detectaron diversos problemas relacionados con malas prácticas de programación, diseño deficiente, ausencia de estándares, falta de revisiones de código, deuda técnica y poca claridad para determinar cuándo una funcionalidad está realmente terminada. Estos problemas han provocado retrasos, aumento en los costos de mantenimiento y dificultad para evolucionar el software.

Desde la perspectiva de la Ingeniería de Software, la calidad no depende únicamente de que el sistema "parezca funcionar", sino de que sea entendible, mantenible, extensible, probado y alineado con las necesidades reales del cliente. Por ello, este informe identifica problemas de calidad, los analiza con base en Clean Code y SOLID, propone acciones para reducir deuda técnica, define un Producto Mínimo Viable (MVP) y establece una Definition of Done (DoD) para mejorar el proceso de desarrollo.

2. Identificación de Problemas

No.	Problema Identificado	¿Qué ocurre en el sistema?	¿Por qué es una mala práctica?	Consecuencias Directas
1	Clases con más de 1,800 líneas	Algunas clases concentran demasiada lógica de negocio.	Rompe la cohesión; una clase tan grande abarca múltiples responsabilidades.	Aumenta el tiempo de mantenimiento y hace riesgoso modificar el código.
2	Funciones con más de 250 líneas	Existen métodos individuales demasiado extensos.	Una función debe realizar una sola tarea concreta; si es muy larga, mezcla subtareas.	Se vuelve sumamente difícil probarla, reutilizarla y depurar fallos.
3	Código copiado y pegado	Se reutiliza código duplicándolo entre diferentes módulos.	Viola de forma directa el principio DRY (Don't Repeat Yourself).	Si hay un error, debe corregirse en varios lugares, provocando inconsistencias.
4	Variables poco descriptivas	Se usan nombres genéricos como x, dato, temp, valor1 u obj.	Los nombres no comunican la intención ni el contexto de la variable.	Otros desarrolladores tardan más en entender el código y cometen errores.
5	Archivos con demasiadas responsabilidades	Un mismo archivo valida datos, consulta la BD, genera reportes y envía correos.	Mezcla capas de la arquitectura (Presentación, Negocio, Datos).	El sistema queda fuertemente acoplado; un cambio rompe funciones ajenas.
6	Estructura inconsistente	Cada desarrollador organiza archivos y carpetas de forma distinta.	La falta de convenciones dificulta navegar e integrar el proyecto.	Se pierde tiempo buscando componentes y se generan duplicidades en el disco.
7	Ausencia de guía de estilos	No existe una convención formal (como PSR en PHP o StandardJS).	Sin reglas comunes, el formato del código queda parchado e inconsistente.	Aumenta la fricción en el equipo y baja la legibilidad general.
8	Comentarios innecesarios	Los comentarios explican qué hace la línea en lugar de por qué se decidió así.	El código debe ser autoexplicativo; comentar todo oculta una mala nomenclatura.	Los comentarios se desactualizan rápido, convirtiéndose en mentiras.
9	Terminado por "percepción visual"	Las funciones se aceptan sólo porque la interfaz gráfica "parece funcionar".	La calidad no debe basarse en pruebas informales de "caja negra" visual.	Se integran errores ocultos al repositorio principal que estallan en producción.

10	Sin revisiones de código	Nunca se realizan Code Reviews o revisiones entre compañeros.	Se pierde la oportunidad de detectar errores tempranos y compartir conocimiento.	Se acumulan defectos y se crean silos de código donde sólo uno le entiende.
11	Efecto dominó en cambios	Cada nueva funcionalidad requiere modificar varias clases existentes.	Es un síntoma claro de alto acoplamiento y bajo diseño modular.	Aumenta drásticamente el riesgo de regresiones (romper lo que ya funcionaba).
12	Scope Creep (Funciones no pedidas)	El equipo agrega funciones extras porque "piensan que serian útiles".	Desarrollar sin validar valor con el cliente desperdicia tiempo y dinero.	Se retrasa el núcleo del sistema y crece el alcance sin control real.
13	Errores conocidos ignorados	Se posponen o ignoran fallas activas para simular que se avanza rápido.	Acumular bugs deliberadamente genera una deuda técnica insostenible.	Los defectos crecen, dañan la base de datos y su corrección futura es más cara.
14	Documentación obsoleta	Los diagramas o manuales iniciales nunca se actualizan.	La documentación pierde total validez si no refleja el estado real del software.	Los nuevos integrantes se confunden y toman malas decisiones técnicas.
15	Sin criterio de aprobación formal	No existe un proceso (Merge Requests / Pull Requests) para integrar código.	Sin controles de calidad, se acepta código incompleto o sin pruebas.	Disminuye la estabilidad de la rama principal del proyecto (main/master).

3. Análisis de Clean Code

3.1 Clases Demasiado Grandes

- **Principio Incumplido:** Cohesión y tamaño de clases (Las clases deben ser pequeñas y enfocadas).
- **Análisis en GymManager:** La existencia de clases con más de 1,800 líneas demuestra que el sistema padece del antipatrón **Objeto Dios** (God Object), centralizando lógica que debería estar repartida en módulos independientes.
- **Corrección Propuesta:** Aplicar refactorización mediante la extracción de clases. Dividir el archivo en controladores delgados, servicios de negocio, repositorios de consulta y validadores de formularios.
- **Beneficios:** Código altamente legible, facilidad para escribir pruebas unitarias automatizadas y reducción del riesgo de colisiones (**merge conflicts**) en Git.

3.2 Funciones Demasiado Largas

- **Principio Incumplido:** Funciones de una sola responsabilidad (Deben hacer una sola cosa y hacerla bien).
- **Análisis en GymManager:** Métodos con más de 250 líneas indican que la función realiza control de flujo, validación, consultas SQL y formateo de respuesta al mismo tiempo, incrementando la complejidad ciclomática.
- **Corrección Propuesta:** Aplicar el patrón **Extract Method** (Extraer Método). Subdividir la función en pequeños bloques de código de no más de 20 líneas con nombres altamente expresivos.
- **Beneficios:** Reutilización de subtareas, facilidad para aislar bugs y métodos **autoexplicativos** que reducen la carga cognitiva del desarrollador.

3.3 Nombres Poco Descriptivos

- **Principio Incumplido:** Nombres con sentido e intención reveladora.
- **Análisis en GymManager:** El uso de `x`, `dato` o `temp` obliga al lector a rastrear toda la función para deducir qué tipo de información almacena la variable.
- **Corrección Propuesta:** Renombrar todas las variables basándose en el dominio del negocio.
 - Malo: `let x = 10;` → Bueno: `let diasMembresiaRestantes = 10;`
 - Malo: `let obj = get();` → Bueno: `let entrenadorAsignado = obtenerEntrenadorActivo();`
- **Beneficios:** El código se convierte en su propia documentación técnica, disminuyendo los tiempos de inducción (onboarding) para nuevos programadores.

3.4 Código Duplicado

- **Principio Incumplido:** Principio DRY (Don't Repeat Yourself).
- **Análisis en GymManager:** El copiar y pegar código genera redundancia. Si las reglas de negocio para calcular el recargo de una membresía vencida cambian, el desarrollador tendrá que buscar y modificar de forma manual cada lugar duplicado.
- **Corrección Propuesta:** Abstraer la lógica común en **Helpers, Traits** o clases de Servicio compartidas que puedan ser invocadas desde cualquier módulo que lo requiera.
- **Beneficios:** Consistencia en todo el sistema. Un cambio se realiza en un solo lugar y se propaga automáticamente, asegurando la integridad del comportamiento del software.

3.5 Comentarios que Explican Cada Línea

- **Principio Incumplido:** Los comentarios no salvan el mal código; el buen código es autoexplicativo.
- **Análisis en GymManager:** Comentar cada paso del código satura visualmente los archivos y usualmente delata que el código está mal estructurado o tiene variables mal nombradas.
- **Corrección Propuesta:** Depurar los comentarios redundantes. Si un comentario dice // incrementa x en 1, se elimina. Si explica una regla fiscal o una decisión técnica compleja del gimnasio, se conserva.
- **Beneficios:** Archivos más limpios y compactos. Evita la confusión provocada por comentarios que dicen una cosa pero hacen otra tras una modificación descuidada.

4. Aplicación de los Principios SOLID

4.1 Principio de Responsabilidad Única (SRP)

Definición: Una clase debe tener una única razón para cambiar.

- **Evidencia de Fallo:** Un solo archivo valida datos, consulta la base de datos, genera reportes y envía correos electrónicos.
- **Solución:** Desacoplar el archivo aplicando una arquitectura en capas:

[Request] → [ValidatorClass] → [GymController] → [PaymentService] → [NotificationMailer]

- **Resultado:** Si el diseño del correo cambia, solo se modifica el Mailer. Si la regla de pago cambia, solo se altera el **Service**.

4.2 Principio Abierto/Cerrado (OCP)

Definición: El software debe estar abierto a la extensión, pero cerrado a la modificación.

- **Evidencia de Fallo:** Cada nueva funcionalidad o tipo de membresía obliga a alterar múltiples clases ya existentes, lo que demuestra rigidez.
- **Solución:** Utilizar abstracciones e interfaces. Por ejemplo, definir una interfaz **MembresiaInterface con un método calcularAcceso()**. Cada nueva modalidad (VIP, Estudiante, Corporativa) implementará la interfaz sin tocar el código central.
- **Resultado:** Escalabilidad segura sin riesgo de romper módulos estables.

4.3 Principio de Sustitución de Liskov (LSP)

Definición: Las subclases o implementaciones deben ser sustituibles por sus clases base sin alterar el comportamiento correcto del programa.

- **Evidencia de Fallo:** La falta de una estructura común e interfaces provoca que un módulo maneje los reportes de ingresos de una forma y el de asistencia de otra totalmente incompatible.
- **Solución:** Crear contratos formales estrictos mediante tipos de datos obligatorios. Si se tiene una clase base **ReporteBase**, cualquier subclase (**ReporteIngresos, ReporteAsistencia**) debe aceptar y retornar las mismas estructuras de datos.
- **Resultado:** Intercambiabilidad perfecta de componentes y predictibilidad del sistema.

4.4 Principio de Segregación de Interfaces (ISP)

Definición: Es mejor tener muchas interfaces específicas que una sola interfaz de propósito general.

- **Evidencia de Fallo:** Clases pesadas obligan a ciertos módulos a implementar o heredar métodos que no necesitan (por ejemplo, que el módulo de asistencia cargue funciones de facturación).
- **Solución:** Dividir los contratos. Crear `InterfaceReporteImprimible`, `InterfaceNotificable` e `InterfaceFacturable` por separado. Las clases solo implementarán lo que realmente vayan a utilizar.
- **Resultado:** Clases más limpias y desacopladas; se evitan dependencias innecesarias de bajo nivel.

4.5 Principio de Inversión de Dependencias (DIP)

Definición: Depender de abstracciones (interfaces), no de implementaciones concretas.

- **Evidencia de Fallo:** El sistema depende directamente del gestor de correo o del motor SQL dentro de la lógica de negocio, impidiendo realizar pruebas eficientes o cambiar de herramientas.
- **Solución:** Implementar **Inyección de Dependencias**. El controlador no debe instanciar directamente una librería externa de correos; debe pedir una abstracción a través de su constructor:

PHP

```
public function __construct(MailServiceInterface $mailer)
```

- **Resultado:** Alta flexibilidad. Permite simular el envío de correos (mocking) en entornos de prueba sin realizar envíos reales, y facilita migrar de proveedor tecnológico en el futuro.

5. Análisis de la Deuda Técnica

5.1 Decisiones que Generan Deuda Técnica

La deuda técnica en GymManager se ha acumulado por priorizar la velocidad de entrega superficial sobre la calidad de la ingeniería de software. Las causas raíz son:

1. **Improvisación de Arquitectura:** Permitir archivos multiusos por falta de directrices claras.
2. **Omisión de Procesos:** Saltarse los Code Reviews y las pruebas unitarias para simular avance rápido.
3. **Desatención a Fallos:** Acumular reportes de bugs abiertos en lugar de resolverlos de inmediato.
4. **Crecimiento Descontrolado:** Desarrollar módulos extra sin la previa validación ni aprobación del cliente.

5.2 Tipo de Deuda Técnica Observada

- **Deuda de Diseño (Arquitectura):** Severa. Mal acoplamiento entre la persistencia de datos y la interfaz de usuario.
- **Deuda de Código:** Alta. Presencia de Code Smells (código maloliente) como funciones gigantes y variables crípticas.
- **Deuda de Proceso:** Crítica. Ausencia total de metodologías ágiles de control de calidad y validación de entregables.

5.3 Impacto en el Mantenimiento

El mantenimiento se volverá una tarea lenta y frustrante. Introducir un cambio menor en el módulo de pagos puede desencadenar fallos ocultos en el módulo de reservación de clases debido al fuerte acoplamiento. Además, el costo económico de corregir un error en fases tardías incrementa de manera exponencial.

5.4 Impacto en Tiempo y Costo

Lo que al principio pareció un "ahorro de tiempo", a mediano plazo duplicará los costos. El equipo gastará el 80% de su tiempo resolviendo regresiones y apagando fuegos en producción, en lugar de programar nuevas características de valor, retrasando el despliegue final.

5.5 Acciones para Disminuir la Deuda Técnica

[Auditoría Actual] → [Plan de Refactorización] → [Guía de Estilos + DoD] → [Estabilización del Sistema]

Acción Propuesta	Implementación Técnica	Resultado Esperado
Refactorización Arquitectónica	Aplicar SRP y extraer clases Dios a Servicios.	Código modular, legible y aislado ante fallos.
Modularización de Funciones	Romper métodos de 250 líneas en funciones atómicas.	Reducción drástica de complejidad; reutilización.
Abstracción del Código Común	Migrar bloques duplicados a Helpers y Traits.	Principio DRY consolidado; cambios centralizados.
Estandarización de Código	Adoptar una Guía de Estilos oficial del lenguaje.	Homogeneidad estética; facilidad de lectura en equipo.
Institución de Code Reviews	Configurar Pull Requests obligatorios con aprobación.	Compartición de conocimiento y freno a malas prácticas.
Automatización de Pruebas	Incorporar una suite mínima de pruebas unitarias.	Red de seguridad técnica antes de subir a producción.
Actualización de Documentación	Documentar APIs y arquitectura viva en Markdown.	Reducción del tiempo de onboarding de desarrolladores.
Establecimiento de una DoD	Aplicar lista de verificación obligatoria por tarea.	Garantía de calidad real antes de dar por cerrada una actividad.
Plan de Saneamiento de Bugs	Detener funciones nuevas hasta mitigar errores conocidos.	Eliminación de pasivos técnicos acumulados.
Gestión de Configuración	Delimitar el alcance estrictamente al MVP acordado.	Eliminación del desperdicio de tiempo y recursos.

6. Definición del MVP (Producto Mínimo Viable)

El MVP debe concentrarse exclusivamente en las operaciones críticas que permiten abrir el gimnasio y cobrar a los clientes. Las funciones sofisticadas se relegan a fases posteriores para no arriesgar la viabilidad del proyecto.

6.1 Funcionalidades que Forman Parte del MVP

[Gestión Clientes] + [Registro Pagos] + [Control Asistencia] → ¡Gimnasio Operativo! (MVP)

- **Gestión de Clientes:** INCLUIR. Es la entidad núcleo; sin usuarios registrados no se puede asociar ninguna otra operación del negocio.
- **Registro de Pagos:** INCLUIR. Vital para la supervivencia económica del gimnasio, permitiendo auditar quién debe y quién está al día.
- **Control de Asistencia:** INCLUIR. Permite validar el acceso en la entrada del establecimiento según el estatus de su pago.
- **Reservación de Clases:** INCLUIR. Necesario si el gimnasio ofrece actividades con aforo limitado (spinning, yoga) para evitar sobrecupos operativos.
- **Reportes Básicos de Ingresos:** INCLUIR. Tablero administrativo indispensable para que el dueño evalúe la rentabilidad diaria del negocio.

6.2 Funcionalidades para Futuras Versiones

- **Chat Interno entre Entrenadores:** EXCLUIR. Se puede solventar externamente (WhatsApp). No aporta valor crítico en la fase de apertura.
- **Integración con Relojes Inteligentes:** EXCLUIR. Complejidad técnica extrema (APIs de Apple) que consume tiempo valioso sin validar el modelo de negocio principal.
- **Sistema de Recompensas (Gamificación):** EXCLUIR. Estrategia de fidelización secundaria. Primero se requiere estabilizar la operación del negocio.
- **Publicación Automática en Redes Sociales:** EXCLUIR. Automatización de marketing no esencial para el control administrativo interno.
- **Tienda en Línea de Suplementos:** EXCLUIR. Añade complejidad de e-commerce (pasarelas de pago extras, inventario físico de productos). Debe ser un proyecto independiente.

6.3 Justificación General del MVP

Al acotar el alcance a estos cinco módulos esenciales, el equipo reduce drásticamente las dependencias tecnológicas complejas. Esto asegura una salida a mercado rápida, permitiendo recolectar feedback real de los usuarios del gimnasio sin haber invertido meses de presupuesto en características accesorias que el cliente final podría no usar.

7. Definition of Done (DoD)

Una funcionalidad en el sistema GymManager se considerará **formalmente terminada ("Done")** únicamente cuando cumpla al 100% con los siguientes criterios de calidad:

[Código Escrito] → [Estilos Validados] → [Pruebas Aprobadas] → [Code Review OK] → [DONE]

1. **Alineación al Requerimiento:** Satisface estrictamente los criterios de aceptación acordados previamente con el cliente en la historia de usuario.
2. **Control de Alcance:** No incluye código, vistas ni funciones huérfanas ajenas a la tarea en cuestión (cero Scope Creep).
3. **Clean Code Aplicado:** Las variables y métodos poseen nombres descriptivos y claros. No existen nombres genéricos.
4. **Modularidad Obligatoria:** Ningún método nuevo excede las 30 líneas de código; las subtarefas fueron extraídas correctamente.
5. **Arquitectura Limpia:** La lógica está correctamente separada en sus capas (Validación, Controlador, Servicio, Modelo).
6. **Principio DRY Consolidado:** Se validó que no exista duplicación de código; se reutilizaron los servicios generales del sistema.
7. **Validación de Estilos:** El código pasa la validación automática de formato conforme a las directrices estéticas del equipo.
8. **Validación de Entradas Robustas:** Todos los formularios validan tipos de datos, obligatoriedad y límites, respondiendo con alertas claras al usuario.
9. **Pruebas Unitarias Exitosas:** La suite de pruebas de la funcionalidad se ejecuta con un resultado de 100% de éxito.
10. **Pruebas Manuales E2E Realizadas:** El desarrollador completó el flujo feliz y los flujos alternos directamente desde la interfaz de usuario.
11. **Ausencia de Regresiones:** Se verificó manualmente que los cambios no afecten ni rompan funcionalidades previas del sistema.
12. **Revisión por Pares (Code Review):** El código fue revisado, comentado y aprobado por al menos un desarrollador homólogo mediante un Pull Request.
13. **Cero Errores Críticos Conocidos:** La funcionalidad no se sube a la rama principal con bugs abiertos de severidad alta o media.
14. **Documentación al Día:** Se actualizaron los archivos Markdown técnicos y los diagramas de entidad-relación en caso de cambios en la base de datos.
15. **Aprobación del Líder Técnica:** El Product Owner o Líder Técnico firma y valida que la característica cumple con el estándar de entrega.

8. Conclusión

El pilar fundamental para el éxito y la sostenibilidad de cualquier proyecto de software radica en el binomio conformado por **las buenas prácticas de diseño (Clean Code / SOLID) y el rigor metodológico (Definition of Done)**. La calidad del software no debe visualizarse como una actividad opcional o un lujo de tiempo; es una inversión obligatoria que previene el colapso operativo del sistema.

El análisis del estado actual de **GymManager** evidencia que avanzar sin controles de calidad claros, permitiendo código espagueti y clases descomunales, genera una falsa ilusión de velocidad que termina devorando el presupuesto en forma de retrabajo y corrección constante de errores.

Al adoptar la estrategia propuesta en esta auditoría—estructurando el código bajo los principios SOLID, delimitando un MVP realista para contener los costos y aplicando una Definition of Done estricta—el equipo de desarrollo no solo estabilizará la plataforma, sino que transformará a GymManager en un producto seguro, robusto, fácil de mantener y listo para escalar con éxito en el mercado.